Name: SRI VIGNESH P
RegNo : 230701339

EXPERIMENT – 7

Performing Code Injection on Application Process Using Ptrace in Kali Linux

Aim

To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

Software Requirements

- **HostOperatingSystem:** Windows / Linux / macOS
- **VirtualizationSoftware:** Oracle VirtualBox
- **GuestOperatingSystem:** Kali Linux
- **SystemRequirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB

Introduction

Code injection is a technique where malicious code is inserted into a running process to alter its execution. ptrace is a Linux system call that allows one process to observe and control another process, commonly used by debuggers.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Log in using Kali credentials.

Step 2: Create a Target Program

1. OpenTerminal.
2. CreateasimpleCprogram:nanotarget.c
3. Writeabasicinfiniteloopprogram.

```
#include <stdio.h>
int main() {
    while(1) {
        printf("This is an infinite loop...\n");
    }
    return 0;
}
```

1. Compile the program:
2. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:
2. ./target
3. Note the process Id (PID)
4. psaux|greptarget

Step 4: Create Injector Program

1. Createinjectorsourcefile:
2. nanoinjector.c
3. Write ptrace-based code to attach to the target process and modify its memory or registers

```
#include          <stdio.h>
#include         <stdlib.h>
#include  <sys/ptrace.h>
#include   <sys/types.h>
#include     <sys/wait.h>
#include     <sys/user.h>
#include <unistd.h>
 int main(int argc, char *argv[]) {

    if (argc != 2) {
        printf("Usage: %s <pid>\n", argv[0]);
        return 1;
    }

    pid_t pid = atoi(argv[1]);
    struct user_regs_struct regs;

    // Attach to target process
    if (ptrace(PTRACE_ATTACH, pid, NULL, NULL) == -1) {
        perror("ptrace attach failed");
```

```
        return 1;
    }

    // Wait for process to stop
    waitpid(pid, NULL, 0);
    printf("Attached to process %d\n", pid);

    // Get registers
    if (ptrace(PTRACE_GETREGS, pid, NULL, &regs) == -1) {
        perror("getregs failed");
        return 1;
    }

    printf("Original RIP: %llx\n", regs.rip);

    // Modify instruction pointer (example: +2 bytes)
    regs.rip += 2;

    if (ptrace(PTRACE_SETREGS, pid, NULL, &regs) == -1) {
        perror("setregs failed");
        return 1;
    }

    printf("Modified RIP: %llx\n", regs.rip);

    // Detach process
    ptrace(PTRACE_DETACH, pid, NULL, NULL);
    printf("Detached from process\n");
    return 0;
}
```

4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:
2. gcc injector.c -o injector

Step 6: Attach to Target Process

1. AttachinjectortotargetprocessusingPID:
2. sudo./injector<PID>
3. Injectorgainscontrolofthetargetprocess.

Step 7: Inject and Execute Code

1. Modify target process memory or registers.
2. Resume execution of the target process.
3. Observe injected behavior.

Output

```
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an infinite loop ...
This is an in^C

┌──(student㉿kali)-[~]
└─$ ps aux | grep target
student    27486  0.0  0.1   6544  2264 pts/0    S+   15:52   0:00 grep --color=auto target

┌──(student㉿kali)-[~]
└─$ nano injector.c

┌──(student㉿kali)-[~]
└─$ nano injector.c

┌──(student㉿kali)-[~]
└─$ gcc injector.c -o injector

┌──(student㉿kali)-[~]
└─$ sudo ./injector 27486
[sudo] password for student:
ptrace attach failed: No such process

┌──(student㉿kali)-[~]
└─$ ps aux | grep target
student    29734 23.1  0.0   2568  1316 pts/2    S+   15:57   0:02 ./target
student    29829  0.0  0.1   6544  2312 pts/0    S+   15:57   0:00 grep --color=auto target

┌──(student㉿kali)-[~]
└─$ sudo ./injector 29734
Attached to process 29734
Original RIP: 7f61a84d1687
Modified RIP: 7f61a84d1689
Detached from process
```

Result

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.